Assignment 1

Design, Analyze and Validate an Efficient Search Strategy for a Large-Scale Student Record System

DESIGN AND ANALYSIS OF ALGORITHMS – ASSIGNMENT 1 (CO1)

CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution.

Name: K. Hameed Basha

Reg No:192425216

Overview: the problem and the plan

A university stores student records, each identified by a unique Registration Number (a 9-digit integer in our experiments). Staff repeatedly ask: "Does this registration number exist, and where is its record?" The database grows from 1,000 to 1,000,000+ records. We must choose a search strategy, prove its efficiency mathematically, validate it by experiment, and then re-evaluate it when the workload becomes 50,000 searches per hour on 1,000,000 records with new records added only once per day.

The central idea of the whole assignment is one formula:

Total cost = Preprocessing cost + (number of searches q) × (cost of one search)

Linear search has no preprocessing but an expensive search. Binary search and hashing pay a one-time preprocessing cost to make every later search cheap. Which one is best depends on n (the number of records) and q (the number of searches) – and that is exactly why the changed requirement in Task 12 matters.

	A. Linear search	B. Binary search	C. Hash table (chaining)
Design strategy	Brute force	Decrease-and-conquer (decrease by a factor of 2)	Hashing – space–time trade-off (input enhancement)

	A. Linear search	B. Binary search	C. Hash table (chaining)
Style	Iterative	Recursive	Iterative (chain traversal)
Preprocessing	None	Sort the records: $\Theta(n \log n)$	Build table: $\Theta(n)$
Basic operation	A[i] = K	K vs A[mid]	keys[i] = K in a chain
Search cost (worst)	n	$\lfloor \log_2 n \rfloor + 1$	n (theory) – about 8 observed
Search cost (average)	(n+1)/2	$\approx \log_2(n+1) - 1$	$1 + \alpha/2 \approx 1.375$
Class	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$ expected, O(n) worst

Task 1 – Design the Algorithm

Three strategies were chosen because they represent three different ways of thinking about the same problem: (A) look at everything, (B) repeatedly throw away half of the data, and (C) compute where the record must be. They differ in design paradigm, preprocessing requirement and growth rate, so they are not variants of one algorithm. (An iterative and a recursive version of binary search would count as one approach, so only the recursive version is used.)

Approach A – Linear (sequential) search [iterative, brute force]

Basic idea. Scan the records one by one from the start; stop when the registration number matches or the records run out. It makes no assumption about the order of the data.

Preprocessing. None – the records can be searched in the order in which they arrived.

Why investigate it. It is the natural baseline and the only method with zero preprocessing cost. For a very small database, or a single one-off search, it can actually be the cheapest option. Every other approach must beat it to justify its preprocessing.

```
ALGORITHM LinearSearch(A[0..n-1], K)
   // Input : array A of n registration numbers (any order), search key K
   // Output: index of K in A, or -1 if K is absent
   for i <- 0 to n-1 do
      if A[i] = K then return i
   return -1
```

Approach B – Binary search [recursive, decrease-and-conquer]

Basic idea. If the records are sorted by registration number, compare K with the middle record. If they are equal we are done; if K is smaller, the record can only be in the left half; otherwise it can only be in the right half. One comparison therefore discards half of the remaining records, and the same problem (half the size) is solved recursively.

Preprocessing. The array must be sorted in ascending order. We use merge sort (itself divide-and-conquer), costing $\Theta(n \log n)$. This cost is included in all later analysis.

Why investigate it. It is the classic way to exploit order. It needs no extra memory, has a guaranteed worst case, and supports ordered operations such as range queries. It also gives a genuine recursive algorithm for the recurrence-relation tasks.

```
ALGORITHM MergeSort(A[0..n-1])              // preprocessing for B
if n <= 1 then return A
L <- MergeSort(A[0 .. n/2 - 1]);  R <- MergeSort(A[n/2 .. n-1])
merge the sorted halves L and R into one sorted array and return it

ALGORITHM BinarySearch(A[lo..hi], K)        // call with lo = 0, hi = n-1
if lo > hi then return -1                   // base case: empty range
mid <- floor((lo + hi) / 2)
if A[mid] = K then return mid
if K < A[mid] then return BinarySearch(A, lo, mid-1, K)
else               return BinarySearch(A, mid+1, hi, K)
```

Approach C – Hash table with separate chaining [iterative, hashing / space–time trade-off]

Basic idea. Use the registration number to compute the address of its record: a hash function h(K) = K mod m maps every key to one of m buckets. Keys that land in the same bucket are linked in a short chain. A search computes h(K) and walks only that one chain.

Preprocessing. Choose a prime table size m ≈ n / 0.75 (load factor $\alpha = n/m \leq 0.75$) and insert all n records, $\Theta(n)$ in total. Extra memory is traded for speed.

Why investigate it. It is the only approach whose expected search cost does not grow with n, and its preprocessing ($\Theta(n)$) is cheaper than sorting. It shows the limit of what is theoretically achievable and makes the trade-offs (memory, worst case, no ordering) visible.

```
ALGORITHM BuildHash(A[0..n-1])              // preprocessing for C
   m <- smallest prime >= n / 0.75
   head[0..m-1] <- -1;  next[0..n-1] <- -1
   for i <- 0 to n-1 do
      h <- A[i] mod m
      next[i] <- head[h];  head[h] <- i      // insert at front of chain

ALGORITHM HashSearch(K)
   i <- head[K mod m]
   while i != -1 do
      if A[i] = K then return i
      i <- next[i]
   return -1
```

Task 2 – Identify the Basic Operation

The basic operation is the operation in the innermost, most frequently executed part of the algorithm; the total running time is proportional to the number of times it runs. For searching, the natural choice is the **key comparison** – but its location differs in each algorithm.

Algorithm	Basic operation and where it occurs	Why it is appropriate	How its count changes as n grows
A. Linear	Comparison A[i] = K in the body of the for-loop.	It runs once per loop iteration; loop-counter updates and the return are proportional to it, so they only change the constant.	Grows in direct proportion to n: up to n comparisons.
B. Binary (recursive)	Comparison of K with A[mid] – one per recursive call, executed before the algorithm decides which half to discard.	Every call does exactly one comparison plus a constant amount of arithmetic (computing mid). The number of calls equals the number of comparisons.	Increases by only 1 each time n doubles ($\approx \log_2 n$).
C. Hash	Comparison A[i] = K while walking the chain of bucket h(K).	Computing K mod m happens once per search (constant), so the only part that varies is the chain length, i.e. the number of key comparisons.	Depends on the load factor $\alpha = n/m$, not on n – stays about 1 to 1.4.
Preprocessing of B	Comparison of two keys inside the merge step of merge sort.	Each merge step does one comparison per element placed.	$\approx n \log_2 n$.
Preprocessing of C	Computing h = A[i] mod m and linking record i into its chain.	Done exactly once for each of the n records.	Exactly n.

Counting key comparisons (rather than seconds) is useful because it is independent of the machine, programming language and compiler; Task 8–10 use both comparison counts and wall-clock time so that theory and practice can be compared.

Task 3 – Derive the Time-Complexity Function

Input size n = number of records. For each approach we count the basic operation, then add the preprocessing cost. Let q be the number of searches performed.

A. Linear search

Worst case. The key is last or absent, so the loop runs for every i from 0 to n−1:

$C_{worst}(n) = \Sigma_{i=0}^{n-1} 1 = n$

Successful search, average case. If the key is equally likely to be at any of the n positions, finding it at position i costs i comparisons (positions counted from 1):

$C_{succ}(n) = (1/n) \cdot \Sigma_{i=1}^{n} i = (1/n) \cdot n(n+1)/2 = (n+1)/2$

With a probability p that the key is present (otherwise all n comparisons are spent):

$C_{avg}(n) = p \cdot (n+1)/2 + (1 - p) \cdot n$

Preprocessing: 0. Total for q searches:

$T_A(n, q) = 0 + q \cdot C(n) \rightarrow q(n+1)/2$ on average for successful searches, $q \cdot n$ worst case

B. Binary search (with sorting)

Worst case. After k comparisons at most $\lfloor n/2^k \rfloor$ records remain, because each comparison discards the middle record and at least half of the others. The search must stop when the range is empty, i.e. when $\lfloor n/2^k \rfloor = 0$, which first happens when $2^k > n$, i.e. $k = \lfloor \log_2 n \rfloor + 1$:

$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$

For n = 1,000; 10,000; 100,000; 1,000,000 this gives 10, 14, 17 and 20 comparisons – exactly what the experiment measured (Task 9).

Average case (successful). Take $n = 2^k - 1$ so the comparison tree is perfectly full: level i has $2^{(i-1)}$ records, each found after i comparisons.

$C_{succ}(n) = (1/n) \cdot \Sigma_{i=1}^{k} i \cdot 2^{(i-1)} = ((k-1) \cdot 2^k + 1) / (2^k - 1) \approx k - 1 + k/n \approx \log_2(n+1) - 1$

An unsuccessful search ends at an empty position at the bottom of the tree, so it costs about $\log_2 n$ (k or k−1 comparisons).

Preprocessing (merge sort). Merging two sorted halves of total length n needs at most n − 1 comparisons, so M(n) = 2M(n/2) + n − 1 with M(1) = 0. Unrolling k times:

M(n) = 2^k · M(n/2^k) + k·n − (2^k − 1) → with k = $\log_2 n$: M(n) = n·$\log_2 n$ − n + 1

Total for q searches:

$T_B(n, q) = (n \cdot \log_2 n - n + 1) + q \cdot (\lfloor \log_2 n \rfloor + 1) = \Theta(n \log n + q \log n)$

C. Hash table with chaining

Let m be the number of buckets and α = n/m the load factor. Assume simple uniform hashing: each key falls into each bucket with probability 1/m.

Unsuccessful search. The whole chain of bucket h(K) is examined. The expected chain length is n/m:

$C_{fail}(n) = \alpha$

Successful search. Records are inserted at the front of the chain, so the i-th inserted record is preceded in its chain only by the records inserted later that fell into the same bucket – on average (n − i)/m of them. Hence the comparisons needed for the i-th record are 1 + (n − i)/m, and averaging over all i:

$C_{succ}(n) = (1/n) \cdot \Sigma_{i=1}^{n} [1 + (n - i)/m] = 1 + (n - 1)/(2m) = 1 + \alpha/2 - \alpha/(2n) \approx 1 + \alpha/2$

With α ≤ 0.75 this is **at most about 1.375 comparisons, whatever the value of n**. In the worst case all n keys fall into one bucket and the search degenerates to linear search: C_worst(n) = n.

Preprocessing. Each of the n records needs one modulo operation and two array assignments (constant work), so Preprocessing = c·n = Θ(n). (Finding the prime m by trial division costs about √m ≈ 1,150 operations for n = 10^6 – negligible.)

$T_C(n, q) = c \cdot n + q \cdot (1 + \alpha/2) = \Theta(n + q)$ *expected*

Overall cost – preprocessing and searching together

Substituting n = 1,000,000 and q = 50,000 searches (all successful, average case) gives the following operation counts:

Approach	Preprocessing (operations)	Searching (operations)	Total (operations)
A. Linear	0	25,000,025,000 (= q·(n+1)/2)	25,000,025,000
B. Binary	18,931,570 (merge sort)	946,579 (≈ 18.9 per search)	19,878,148
C. Hash	1,000,000 (n insertions)	68,750 (≈ 1.375 per search)	1,068,750

So at this load, sorting and hashing cost roughly 1,258× and 23,392× less than repeated linear search, even after paying for preprocessing.

Task 4 – Best, Average and Worst-Case Behaviour

The three cases depend on **where the key is** (or whether it is present at all) – not on the size of the input alone.

A. Linear search

- **Best case – 1 comparison:** the key is the very first record. The loop exits immediately; Θ(1).
- **Average case – (n+1)/2 comparisons:** if the key is present and every position is equally likely, on average half of the list is scanned. If absent keys are also queried, the average moves towards n (formula in Task 3).
- **Worst case – n comparisons:** the key is the last record **or is not in the database at all**. The loop has no early exit and examines every record; Θ(n). The "absent key" case matters in practice because staff often check records that do not exist.

B. Binary search

- **Best case – 1 comparison:** the key sits exactly at the middle position of the sorted array, found in the first call.
- **Average case – ≈ $\log_2(n+1) - 1$ comparisons:** most records lie deep in the comparison tree (half of all records are at its bottom level), so the average is only about one comparison less than the maximum.

- **Worst case – $\lfloor \log_2 n \rfloor + 1$ comparisons:** the key is at the deepest level of the tree or is absent; the range must shrink all the way to empty. Importantly, **the input order of the data cannot make binary search worse** – the bound depends only on n.

C. Hash table

- **Best case – 1 comparison** (0 for an absent key whose bucket is empty): the key is at the front of its chain. About 47% of the buckets were empty in our experiments, so many absent keys cost zero comparisons.
- **Average case – $1 + \alpha/2 \approx 1.375$ comparisons (found), $\alpha \approx 0.75$ (not found):** the keys spread evenly, so chains are short (Task 3).
- **Worst case – n comparisons:** every key hashes to the same bucket (for example, a poor hash function that interacts badly with the numbering pattern of registration numbers, such as K mod 1000 on numbers with fixed trailing digits). The structure collapses into one long list. Using a prime m and checking the longest chain after building the table protects against this; the longest chain observed was 5, 5, 6 and 8 for the four dataset sizes.

Measured evidence at n = 1,000,000 (average of 200 found and 200 absent keys; best/worst for the specially chosen keys):

Case	A. Linear	B. Binary	C. Hash
Best case (comparisons)	1	1	1
Average, key found	521,308	19.16	1.37
Key absent	1,000,000	19.93	0.77
Worst case observed	1,000,000	20	5

Task 5 – Asymptotic Analysis

Definitions used: $f(n) \in O(g(n))$ if $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$; $f(n) \in \Omega(g(n))$ if $f(n) \geq c \cdot g(n)$; $f(n) \in \Theta(g(n))$ if both hold.

Algorithm	Search – worst	Search – average	Search – best	Preprocessing	Total (q searches)
A. Linear	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	none	$\Theta(q \cdot n)$
B. Binary	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(n \log n)$	$\Theta(n \log n + q \log n)$
C. Hash	$O(n)$	$\Theta(1)$ expected (α constant)	$\Theta(1)$	$\Theta(n)$	$\Theta(n + q)$ expected

Justification of the notation

- **Linear:** $C_worst(n) = n$, so with $c_1 = c_2 = 1$ and $n_0 = 1$ it is $\Theta(n)$. The best case is a single comparison, so the running time over all inputs lies between $\Omega(1)$ and $O(n)$; the overall bound that holds for every input is $O(n)$.
- **Binary:** $\lfloor \log_2 n \rfloor + 1 > \log_2 n$ for every $n \geq 1$, and $\lfloor \log_2 n \rfloor + 1 \leq 2 \cdot \log_2 n$ for every $n \geq 2$. So $\log_2 n \leq C_worst(n) \leq 2 \cdot \log_2 n$ ($c_1 = 1$, $c_2 = 2$, $n_0 = 2$), hence $\Theta(\log n)$.
- **Hash:** the expected cost $1 + \alpha/2$ is bounded by the constant 1.375 for any n provided the load factor is kept ≤ 0.75, so it is $\Theta(1)$ in expectation. Without a bound on the worst chain, only $O(n)$ can be guaranteed.
- **Order of growth:** $\lim_{n \to \infty} (\log n)/n = 0$ (L'Hôpital), so $\log n \in o(n)$: binary search grows strictly slower than linear search, and 1 grows slower than both.

What this means for the university

When the database grows 1,000 times (1,000 → 1,000,000 records):

- **Linear search** needs about 1,000 times more work: average comparisons rise from 475 to 521,308 and a failed search from 25.9 µs to 22.6 ms. It stops being practical – each search becomes visible to the user.
- **Binary search** needs only about 2× more work (10 → 20 comparisons): 1.03 µs → 2.54 µs. A thousand-fold larger database costs just ten more comparisons.
- **Hash search** needs about the same work (about 1.4 comparisons both times). Timings stay at a fraction of a microsecond (0.11 µs → 0.14 µs).

The price is paid in preprocessing: Θ(n log n) for B and Θ(n) for C, a one-time cost that must be justified by enough searches (Task 3 and Task 12).

Task 6 – Formulate the Recurrence Relation

The recurrence is derived from the recursive binary search algorithm of Task 1, by reading off what one call does.

Component	In recursive BinarySearch(A, lo, hi, K)
Input size of the original problem	n = hi − lo + 1, the number of records still under consideration.
Size of the recursive sub-problem	The call is made on A[lo..mid−1] or A[mid+1..hi]. Each has at most ⌊n/2⌋ records, since mid is the middle position.
Number of recursive calls	One. Although the code has two recursive calls written, only one of them is executed in any invocation (an if/else choice). This is why the recurrence has T(n/2) once, not twice.
Work outside the recursive call	One comparison of K with A[mid] plus constant arithmetic for mid: Θ(1). We count 1.
Base case	lo > hi (empty range): returns −1 without any comparison, T(0) = 0. Equivalently, T(1) = 1 for a single record.

Putting the components together – (cost of the recursive call on the smaller half) + (cost of the work done in this call):

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \qquad T(1) = 1$$

Where the pieces come from: T(⌊n/2⌋) is the one recursive call on at most half the data, and "+ 1" is the single key comparison per call. For n a power of 2 this simplifies to T(n) = T(n/2) + 1, the form used in the Master Theorem below.

Task 7 – Solve the Recurrence Relation

Three methods are shown. They agree, which also checks the working.

Method 1 – Backward substitution (iteration)

This method is applicable because the recurrence reduces the problem by the same factor each time, so the pattern is easy to see. Take n = 2^k so n/2^j is always an integer:

```
T(n) = T(n/2) + 1
   = [ T(n/4) + 1 ] + 1      = T(n/4) + 2
   = [ T(n/8) + 1 ] + 2      = T(n/8) + 3
   ...
   = T(n/2^k) + k

Stop when n/2^k = 1, i.e. k = log2 n, and use T(1) = 1:
T(n) = T(1) + log2 n = log2 n + 1
```

Hence $T(n) = \log_2 n + 1 \in \Theta(\log n)$.

Method 2 – Master Theorem

The Master Theorem applies to recurrences $T(n) = a \cdot T(n/b) + f(n)$ with $a \geq 1$, $b > 1$. Here **$a = 1$, $b = 2$, $f(n) = 1 = \Theta(n^0)$**.

$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$

Since $f(n) = \Theta(n^{(\log_b a)}) = \Theta(1)$, **Case 2** applies: $T(n) = \Theta(n^{(\log_b a)} \cdot \log n) = \Theta(\log n)$.

Method 3 – Induction on the exact (floor) form

Claim: for the recurrence $T(n) = T(\lfloor n/2 \rfloor) + 1$ with $T(1) = 1$, $T(n) = \lfloor \log_2 n \rfloor + 1$ for all $n \geq 1$.

- **Base:** $T(1) = 1 = \lfloor \log_2 1 \rfloor + 1$.
- **Step:** for $n \geq 2$, assume the claim for all smaller values. Then $T(n) = T(\lfloor n/2 \rfloor) + 1 = (\lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 1) + 1$. For $n \geq 2$ we have $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = \lfloor \log_2 n \rfloor - 1$, so $T(n) = \lfloor \log_2 n \rfloor - 1 + 2 = \lfloor \log_2 n \rfloor + 1$. ■

This proves the exact worst-case formula used in Task 3 and verified experimentally.

Interpretation

The result Θ(log n) means that each recursive call removes half of the remaining records, so the depth of recursion is only $\log_2 n$. At n = 1,000,000 the recursion goes at most 20 calls deep (so there is no risk of stack overflow), and if the database grows by a further factor of 1,000 to a billion records, only about 10 more calls are needed.

> **Preprocessing recurrence.** For M(n) = 2M(n/2) + n − 1: a = 2, b = 2, f(n) = n − 1 ∈ Θ(n) = Θ(n^($\log_2$ 2)), so again Master Theorem Case 2 gives **M(n) = Θ(n log n)**, consistent with the exact value $n \cdot \log_2 n - n + 1$ derived in Task 3.

Task 8 – Implement and Test the Algorithms

All three algorithms, the merge-sort preprocessing, and the test harness were implemented in Python 3 (complete code in Appendix A). The implementation mirrors the pseudocode of Task 1 line for line: the same loop for A, the same recursive function for B and the same head/next chain structure for C.

Test setup (kept identical for all algorithms)

- **Dataset sizes:** n = 1,000; 10,000; 100,000; 1,000,000.
- **Data:** n unique random 9-digit registration numbers, generated with a fixed seed (2026) so that every run produces the same data. Algorithm A and C use the data in arrival (unsorted) order; algorithm B uses the sorted copy produced by its preprocessing.
- **Queries:** the same 400 queries for each algorithm and each n – 200 registration numbers that exist and 200 that do not.
- **Two versions of each function:** an instrumented version that counts key comparisons, and an un-instrumented version used only for timing, so the counter does not distort the timings.
- **Timing method:** time.perf_counter(); the average time over all queries (repeated 40 times for the fast algorithms, once for linear search at n ≥ 100,000 because each search is slow). Machine: Linux-6.18.44-fc-v114-x86_64-with-glibc2.39, Python 3.12.3, single core.

Test cases and correctness

Test case	Expected result	Result
Key at the first position of the unsorted data	Linear returns index 0 after 1 comparison	Passed (all n)
Key at the last position of the unsorted data	Linear returns n−1 after n comparisons	Passed (all n)
Smallest and largest key in the sorted data	Binary search returns 0 and n−1	Passed (all n)
Middle key of the sorted data	Binary search finds it in exactly 1 comparison	Passed (all n)
Key smaller than all records / larger than all records	All three return −1	Passed (all n)
200 random existing keys	All three return an index whose record equals the key	Passed (all n)
200 random absent keys	All three return −1	Passed (all n)
Agreement check	A, B and C agree with Python's set membership on every query	Passed: 1,600 queries, 0 mismatches

Reproducibility: save the code of Appendix A as search_lab.py and run python3 search_lab.py. The seeded data and queries are identical on every run; comparison counts are exactly reproducible, while timings vary slightly with the machine and system load.

Task 9 – Record Experimental Performance

Table 1 – Average key comparisons per search

n	Linear (found)	Linear (absent)	Binary (found)	Binary (absent)	Binary worst	Hash (found)	Hash (absent)	Hash longest chain
1,000	475	1,000	9.04	9.99	10	1.34	0.67	5
10,000	5,093	10,000	12.51	13.41	14	1.42	0.75	5
100,000	44,523	100,000	15.66	16.70	17	1.40	0.77	6
1,000,000	521,308	1,000,000	19.16	19.93	20	1.37	0.77	8

Table 2 – Average execution time per search

n	Linear (found)	Linear (absent)	Binary (found)	Binary (absent)	Hash (found)	Hash (absent)
1,000	0.01 ms	0.03 ms	0.87 µs	1.03 µs	0.106 µs	0.098 µs
10,000	0.11 ms	0.16 ms	1.07 µs	1.18 µs	0.097 µs	0.105 µs
100,000	0.73 ms	2.08 ms	1.86 µs	3.56 µs	0.197 µs	0.180 µs
1,000,000	11.3 ms	22.6 ms	2.44 µs	2.54 µs	0.143 µs	0.131 µs

Table 3 – Preprocessing cost

n	Merge sort – comparisons	$n \cdot \log_2 n - n + 1$ (theory)	Merge sort – time (own code)	Built-in sorted() – time	Build hash table – time	Hash buckets m
1,000	8,689	8,967	1.84 ms	0.10 ms	0.16 ms	1,361
10,000	120,324	122,878	15.9 ms	1.23 ms	0.59 ms	13,337

n	Merge sort – comparisons	$n \cdot \log_2 n - n + 1$ (theory)	Merge sort – time (own code)	Built-in sorted() – time	Build hash table – time	Hash buckets m
100,000	1,536,360	1,560,965	201.3 ms	15.2 ms	12.3 ms	133,337
1,000,000	18,674,774	18,931,570	3.50 s	294.1 ms	283.9 ms	1,333,357

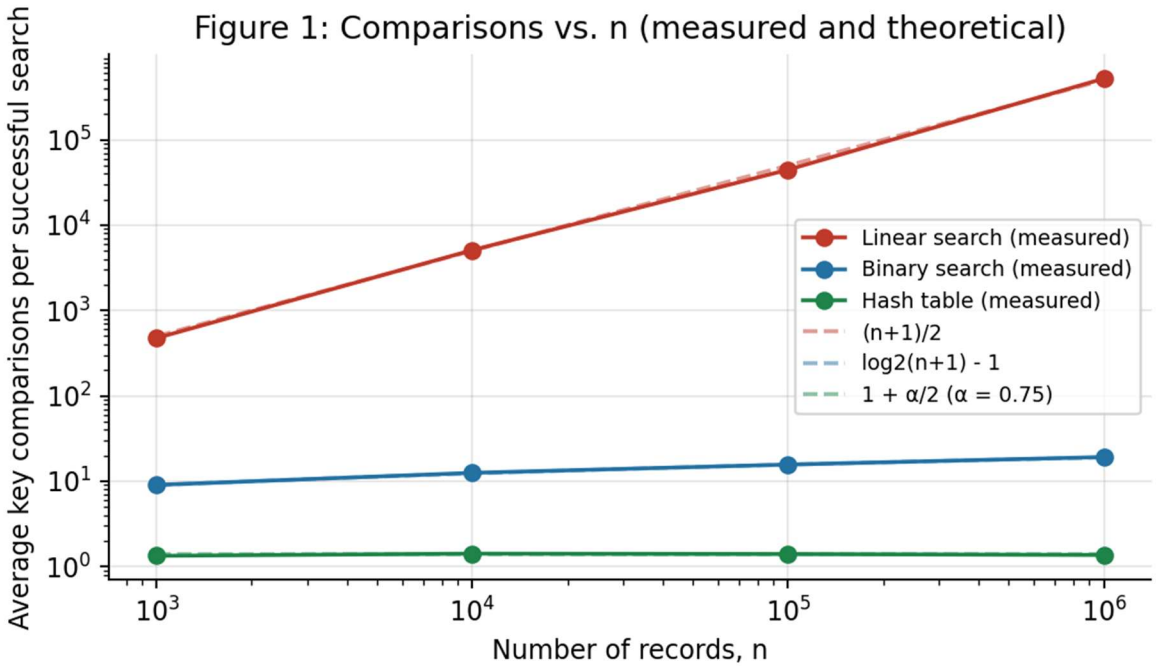


Figure 1 – Average comparisons (found) against n on log–log axes; dashed lines are the theoretical formulas.



Figure 2 – Average time per search. Linear search climbs steeply, binary search rises slowly, hashing is nearly flat.

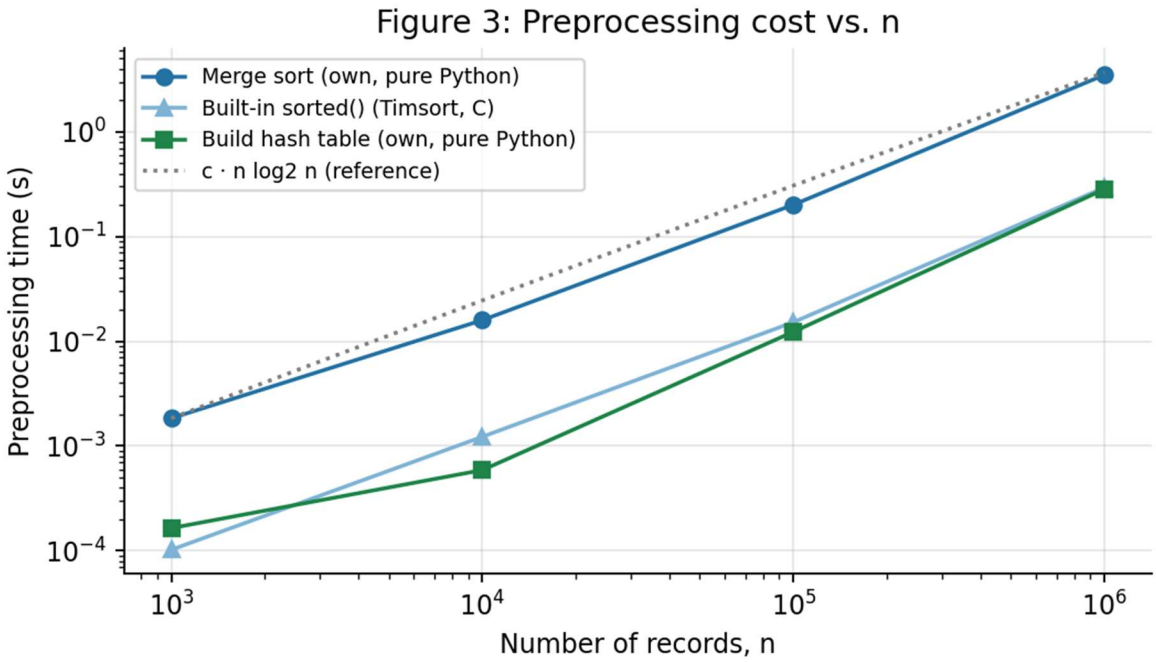


Figure 3 – Preprocessing time. Merge sort follows n log n; hash-table construction is cheaper.

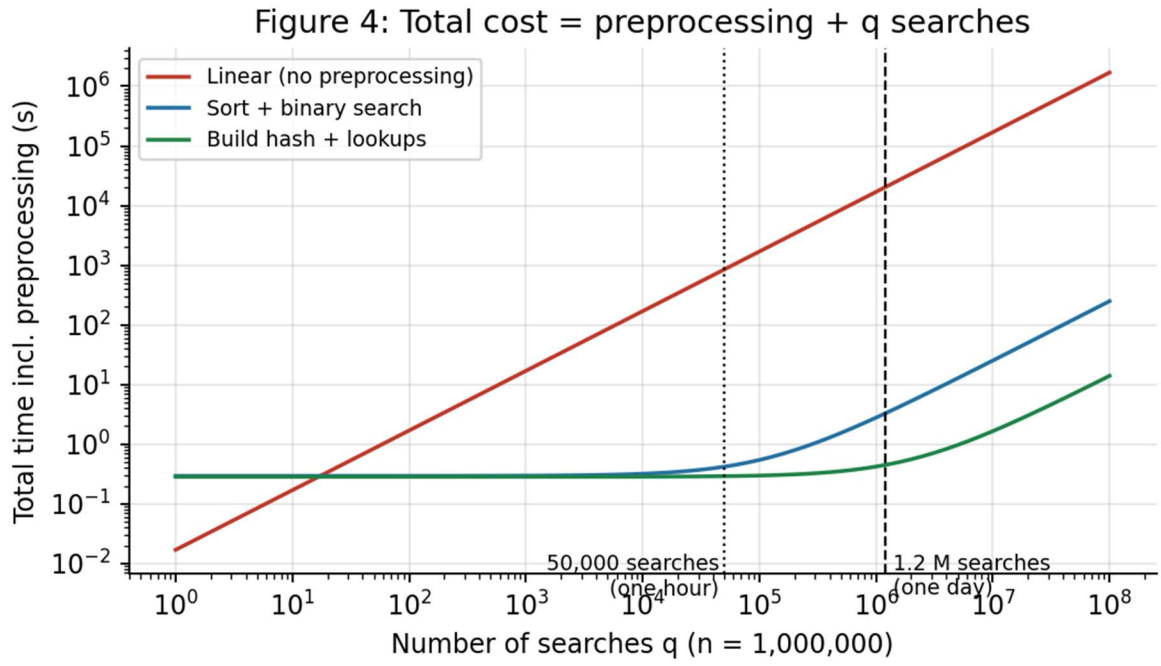


Figure 4 – Total cost (preprocessing + q searches) at n = 1,000,000. Linear search is only cheaper for the first few searches.

Task 10 – Compare Theory with Experiment (4 marks)

Quantity	Theory	Observed at n = 1,000 → 1,000,000	Agreement
Linear: comparisons (found)	(n+1)/2	475 vs 500; 5,093 vs 5,000; 44,523 vs 50,000; 521,308 vs 500,000	Within about 5% at three sizes (−11% at n = 100,000); a 200-query sample has about ±4% sampling error
Linear: comparisons (absent)	n	Exactly n at every size	Exact
Binary: worst-case comparisons	$\lfloor \log_2 n \rfloor + 1$	10, 14, 17, 20 vs 10, 14, 17, 20	Exact

Quantity	Theory	Observed at n = 1,000 → 1,000,000	Agreement
Binary: comparisons (found)	$\approx \log_2(n+1) - 1$	9.04, 12.51, 15.66, 19.16 vs 8.97, 12.29, 15.61, 18.93	Within about 0.2 comparisons
Hash: comparisons (found)	$1 + \alpha/2 \approx 1.375$	1.34, 1.42, 1.40, 1.37	Within 3%; independent of n
Hash: comparisons (absent)	$\alpha \approx 0.75$	0.67, 0.75, 0.77, 0.77	Within about 0.1 (200-query sample)
Merge sort comparisons	$n \cdot \log_2 n - n + 1$ (upper bound)	96.9%, 97.9%, 98.4%, 98.6% of the bound	Always just below the bound

Execution time follows the same ordering. For each 10× increase in n, linear search time grew by about 9.5×, 6.7× and 15.4× (theory: 10× each); from 1,000 to 1,000,000 records binary search time grew only 2.8× (theory: 2.1× in comparisons) and hash time stayed within a factor of 2.0.

Which algorithm scales better?

Hashing scales best, then binary search, then linear search. At n = 1,000,000 a found record costs 11.3 ms with linear search, 2.44 µs with binary search and 0.143 µs with hashing – ratios of about 4,640:1 and 17:1 between successive approaches. Because the preprocessing for B and C is only 294.1 ms and 283.9 ms at $n = 10^6$, the preprocessing is recovered after only about 17 searches compared with linear search (using Python's built-in sort).

Differences between prediction and measurement, and why

- **Constant factors do not appear in Big-O.** Theory says binary search needs 2.1× more comparisons at 10^6 than at 10^3; the measured time grew 2.8×. Large arrays do not fit in the CPU cache, so the later probes of a search (the pointer jumps into distant memory) are slower than the early ones.
- **Linear search at $n = 10^6$ is slower than linear in n.** Its time grew 15.4× for a 10× increase in n. Scanning a million scattered Python integer objects causes cache misses, which small

arrays do not experience. The comparison counts (which are not affected by caches) are exactly linear.

- **Hash construction is not perfectly linear in practice.** Time rose 23.0× from 10^5 to 10^6, because inserting each key writes to a random position of a large array (cache misses). Merge sort time rose 17.4× (theory ≈ 12×) for the same reason plus memory allocation for list slices. Asymptotically both remain $\Theta(n \log n)$ and $\Theta(n)$ in the RAM model that Big-O assumes.
- **Python overhead.** Every Python operation costs far more than the machine instruction it would be in C. This changes absolute times but not the growth rates, which is the point of asymptotic analysis. Python's built-in sorted() is written in C, which is why it is about 12× faster than our pure-Python merge sort.
- **Sampling error and timer noise.** Only 200 found-key queries were drawn per size, so the average position of the key is within a few percent of (n+1)/2, not exactly equal (it was 11% low at n = 100,000, which is about 2.7 standard errors). At sub-microsecond scales, timer resolution and system activity add noise to the hash timings (e.g. $n = 10^4$ is slightly faster than $n = 10^3$).

> **Conclusion.** The experiments confirm the theory: operation counts match the derived formulas almost exactly, and time follows the same ordering and shape. The deviations are explained by memory hierarchy, language overhead and sampling – none contradicts the asymptotic analysis.

Task 11 – Recommend and Defend the Final Design

Recommendation for the original problem (update frequency and search rate unspecified): Approach B – binary search on a sorted array (with merge sort or an equivalent as preprocessing). Approach C (hashing) is the strongest runner-up and is adopted in Task 12 once the workload is known. Linear search is rejected for large databases.

Why B rather than A or C

- **Versus linear search:** at $n = 10^6$ binary search is about 4,640 times faster per search (2.44 µs vs 11.3 ms), and its preprocessing is recovered after about 40 searches by the comparison-count analysis (about 17 with the built-in sort in our timings). It scales logarithmically; linear search does not.

- **Deterministic guarantee:** at most $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons for 10^6 records for any data – no hash function, no assumption about the keys' distribution. Hashing offers only an expected bound; its worst case is O(n) (Task 4).
- **No extra memory:** the sorted array is the database. The hash index needs two extra arrays (1,333,357 bucket heads plus 1,000,000 chain links ≈ 18.7 MB of pointers at $n = 10^6$).
- **Ordered operations:** a sorted array also answers "all students between registration numbers X and Y", "next registration number", and ordered listings, which a hash table cannot do.
- **Honest limitation:** the measured speed of hashing is better than that of binary search for single-key lookups (Table 2), and hash construction is cheaper than sorting. B is preferred for the original problem because of its guarantees, simplicity and ordered operations – not because it is faster.

Efficiency, preprocessing and scalability

Measure (n = 1,000,000)	Value for B
Theoretical search cost	$\Theta(\log n)$: at most 20 comparisons
Experimental search cost	19.2 comparisons on average; 2.44 µs per search
Preprocessing (theory)	$n \cdot \log_2 n - n + 1 = 18{,}931{,}570$ comparisons, $\Theta(n \log n)$
Preprocessing (measured)	294.1 ms (built-in sort) or 3.50 s (own pure-Python merge sort)
Growth to 10^9 records	Only 10 more comparisons (30 instead of 20), but sort cost grows to ≈ n log n

Trade-offs and limitations

- The array must stay sorted. Inserting a single record into a sorted array costs O(n) (shifting elements), so a system with frequent inserts would prefer a balanced tree or hash table; a system with batch updates can merge new sorted records in O(n + k).
- Sorting cost $\Theta(n \log n)$ grows faster than n, but it is paid once and is only a few seconds even for 10^6 records.

- The practicality of B as the database grows is excellent: it remains within microseconds at 10^6 and would need only about 30 comparisons at 10^9 records.

Task 12 – Re-evaluate and Adapt the Design

Assumption for the timings below: half of the queries find a record and half do not (the result is not sensitive to this). Per-search times at n = 10^6 (Table 2): linear 16.9 ms, binary 2.49 µs, hash 0.137 µs.

A. Impact analysis

- **What has changed:** the workload is now fully specified. n is fixed at 10^6, there are **50,000 searches per hour** (≈ 14 per second, 1.2 million per day), and records are added **once per day**. The original problem gave neither the number of searches nor the update pattern.
- **Assumptions affected:** (1) the number of searches q was left open, so we could not tell whether preprocessing would pay off – now it is known to be large; (2) the original design had to allow for records arriving at any time (which is where sorted arrays and static hash tables are weak) – now updates are a daily batch.
- **Why search frequency matters:** Total cost = Preprocessing + q × Search. With q = 1.2 million per day, a cost of even a few milliseconds per search adds up to hours of CPU time, while preprocessing is paid only once. Break-even against linear search is only about 17 searches (built-in sort) – the university exceeds that in under a second.
- **Why rare updates matter:** an index needs rebuilding only when the data change. With one change a day, the whole preprocessing cost is spread over 1.2 million searches: 0.25 µs per search for sorting and 0.24 µs for the hash table – so even an expensive Θ(n log n) rebuild is effectively free, and the weaknesses of both structures (O(n) insertion for the sorted array; resizing and rehashing for the hash table) are not triggered during the day.

B. Evaluation of the original design (2 marks)

Daily workload (n = 10^6, 1.2 M searches)	A. Linear	B. Binary (original)	C. Hash
Preprocessing per day	0	294.1 ms	283.9 ms
Time per search (average)	16.9 ms	2.49 µs	0.137 µs

Daily workload ($n = 10^6$, 1.2 M searches)	A. Linear	B. Binary (original)	C. Hash
CPU time for 50,000 searches in one hour	847 s (23.5% of one core)	0.125 s (0.0035%)	0.0068 s (0.00019%)
Total CPU time per day (incl. preprocessing)	20,320 s (≈ 5.6 hours)	3.28 s	0.45 s

- **Original design under 50,000 searches/hour:** binary search needs only 0.12 seconds of CPU time per hour and each search takes microseconds. It comfortably meets the requirement; there is no performance emergency.
- **Preprocessing acceptable?** Yes. Sorting 10^6 records takes 294.1 ms (or 3.50 s with our slower pure-Python merge sort) once a day – a negligible fraction of 86,400 seconds. A smarter daily update sorts only the k new records and merges them into the existing sorted array in O(n + k).
- **Could another approach do better overall?** Yes, the hash table: daily total 0.45 s against 3.28 s for sorted binary search (about 7.3× better), because its per-search cost is about 18× lower and its construction is cheaper than sorting. Linear search is clearly ruled out: it would need about 5.6 hours of CPU every day and 16.9 ms per search, and would saturate several CPU cores if the database grew ten-fold (10^7 records → 50,000 × 169.3 ms ≈ 2.4 cores).

> **Honest assessment.** Both B and C are fast enough for this workload by a margin of roughly 1,000 : 1 or more. The decision between them is about headroom, latency and robustness, not about whether the requirement can be met.

C. Decision: MODIFY the design (2 marks)

Decision: keep the sorted array (binary search) as the master copy, but **add a hash index and use it for the day-to-day lookups**. This is a modification, not a redesign: the original data structure, its code and its guarantees are all retained.

Revised strategy – a nightly batch rebuild:

- **Step 1 (nightly):** read the day's new records, sort them (O(k log k)) and merge them into the sorted master array in O(n + k).
- **Step 2 (nightly):** rebuild the hash index from the master array with m = the smallest prime ≥ n/0.75, in Θ(n).
- **Step 3 (nightly):** measure the longest chain; if it exceeds a threshold (e.g. 12), choose the next prime for m and rebuild. This turns the theoretical O(n) worst case into a checked, bounded cost (the observed maximum was 8 at $n = 10^6$).
- **Step 4:** build the new index beside the old one and switch over atomically so that no search ever sees a half-built structure.
- **During the day:** every identifier lookup uses the hash index – expected Θ(1) with about 1.4 comparisons. Range queries, ordered reports and an independent verification use binary search on the sorted master array (Θ(log n)).

What changes and why:

- The primary lookup path changes from binary search to hashing, because with 1.2 million searches per day the lower per-search cost dominates.
- The weaknesses that made hashing risky in Task 11 disappear: no insertions during the day means no resizing or rehashing; the table can be built at exactly the right size; and chain lengths are checked at build time.
- The sorted array stays, so ordered queries and a deterministic fallback remain available.

Effect on complexity (per day, $n = 10^6$, $q = 1.2 \times 10^6$):

Before: $T = \Theta(n \log n) + q \cdot \Theta(\log n)$ *After:* $T = \Theta(n \log n)$ *[or* $\Theta(n + k)$ *merge]* $+ \Theta(n) + q \cdot \Theta(1)$

Measured: 3.28 s → 0.45 s per day. The cost of the extra index is about 18.7 MB of memory and the Θ(n) rebuild each night.

D. Evidence-based defence (2 marks)

Evidence	What it shows
Mathematical complexity	Total cost = P(n) + q·S(n). A: 0 + q·Θ(n); B: Θ(n log n) + q·Θ(log n); C: Θ(n) + q·Θ(1). For large q the per-search term dominates, so C < B << A.

Evidence	What it shows
Search frequency	$q = 1.2 \times 10^6$ per day. Break-even against linear search is only about 17–40 searches (measured / by comparison counts), so both preprocessed designs pay off almost immediately.
Preprocessing cost	Once a day: sort 294.1 ms (merge-sort alternative 3.50 s), hash build 283.9 ms. Spread over 1.2 M searches it adds only 0.25 µs and 0.24 µs per search.
Dataset size	At $n = 10^6$ the experiments show 19.2 comparisons (binary) versus 1.4 (hash) versus about 521,000 (linear) for a successful search; the gap widens as n grows (Figure 1).
Experimental results	Per search: linear 16.9 ms, binary 2.49 µs, hash 0.137 µs. Daily CPU total: 20,320 s, 3.28 s, 0.45 s.
Trade-offs	Hash index: +≈ 19 MB memory, no ordered queries by itself, expected (not guaranteed) constant time – all mitigated by keeping the sorted master array, checking chain lengths at build time, and rebuilding in batch.